

Ex. No: 11 Memory Allocation Methods

PROGRAM A: FIRST FIT

C PROGRAM

```
#include <stdio.h>

int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb, np, i, j;
    printf("Enter Number of Blocks: ");
    scanf("%d",&nb);
    printf("Enter Number of Processes: ");
    scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d",&processSize[i]);
    for(i=0;i<np;i++)
    {
        allocation[i]=-1;
        for(j=0;j<nb;j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i]=j;
                blockSize[j]-=processSize[i];
                break;
            }
        }
    }
}
```

```

}
}

printf("\nProcess No\tProcess Size\tBlock No\n");

for(i=0;i<np;i++)
{
printf("%d\t\t%d\t\t",i+1,processSize[i]);

if(allocation[i]!=-1)

printf("%d\n",allocation[i]+1);

else

printf("Not Allocated\n");

}

return 0;

}

```

Output:

```

(kali㉿kali)-[~]
└─$ ./firstfit
Enter Number of Blocks:
5
Enter Number of Processes: 4
Enter Block Sizes:
100 500 200 300 600
Enter Process Sizes:
212 417 112 426

Process No    Process Size    Block No
1             212             2
2             417             5
3             112             2
4             426             Not Allocated

```

SHELL SCRIPT

```

#!/bin/bash

echo "First Fit Memory Allocation Demonstration"

blocks=(100 500 200 300 600)

processes=(212 417 112 426)

echo "Memory Blocks: ${blocks[@]}"

echo "Processes: ${processes[@]}"

echo "Allocation Performed Using First Fit"

```

Output:

```

(kali㉿kali)-[~]
$ ./firstfit.sh
First Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using First Fit

```

PROGRAM B: BEST FIT

C PROGRAM

```
#include <stdio.h>

int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb,np,i,j,bestIdx;
    printf("Enter Number of Blocks: ");
    scanf("%d",&nb);
    printf("Enter Number of Processes: ");
    scanf("%d",&np);
    printf("Enter Block Sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d",&processSize[i]);
    for(i=0;i<np;i++)
        allocation[i]=-1;
    for(i=0;i<np;i++)
    {
        bestIdx=-1;
        for(j=0;j<nb;j++)
```

```

{
if(blockSize[j] >= processSize[i])
{
if(bestIdx==-1 || blockSize[j] < blockSize[bestIdx])
bestIdx=j;
}
}if(bestIdx!=-1)
{
allocation[i]=bestIdx;
blockSize[bestIdx]-=processSize[i];
}
}

printf("\nProcess No\tProcess Size\tBlock No\n");
for(i=0;i<np;i++)
{
printf("%d\t%d\t",i+1,processSize[i]);
if(allocation[i]!=-1)
printf("%d\n",allocation[i]+1);
else
printf("Not Allocated\n");
}
return 0;
}

```

Output:

```

(kali㉿kali)-[~]
└─$ ./bestfit
Enter Number of Blocks: 4
Enter Number of Processes: 3
Enter Block Sizes:
100 200 300 400 500 600
Enter Process Sizes:
212 321 453

Process No      Process Size    Block No
1               500            Not Allocated
2               600            Not Allocated
3               212             3

```

SHELL SCRIPT

```
#!/bin/bash

echo "Best Fit Memory Allocation Demonstration"

blocks=(100 500 200 300 600)

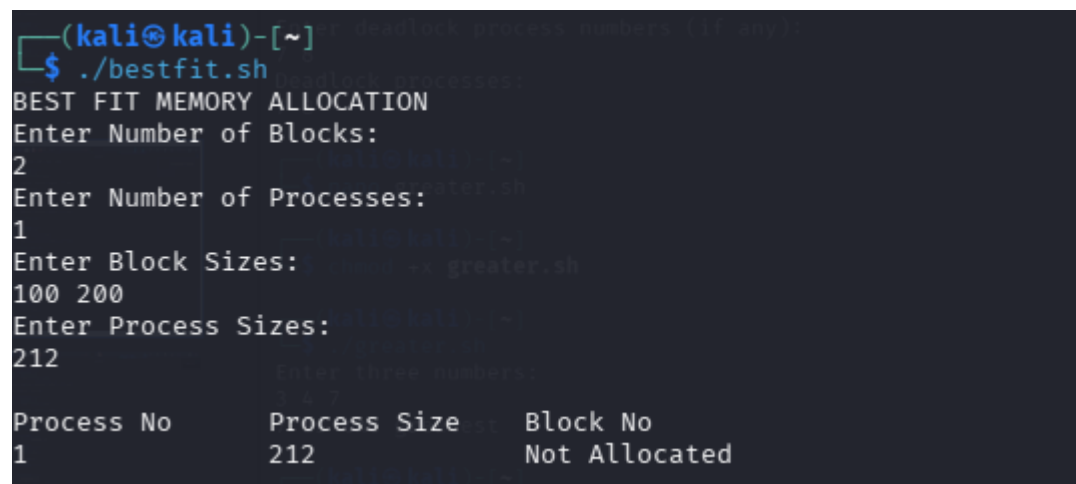
processes=(212 417 112 426)

echo "Memory Blocks: ${blocks[@]}"

echo "Processes: ${processes[@]}"

echo "Allocation Performed Using Best Fit"
```

Output:



```
(kali㉿kali)-[~]
└─$ ./bestfit.sh
BEST FIT MEMORY ALLOCATION
Enter Number of Blocks:
2
Enter Number of Processes:
1
Enter Block Sizes:
100 200
Enter Process Sizes:
212
Process No      Process Size    Block No
1               212           Not Allocated
```

PROGRAM C: WORST FIT

C PROGRAM

```
#include <stdio.h>

int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb,np,i,j,worstIdx;

    printf("Enter Number of Blocks: ");
    scanf("%d",&nb);

    printf("Enter Number of Processes: ");
    scanf("%d",&np);
```

```

printf("Enter Block Sizes:\n");
for(i=0;i<nb;i++)
scanf("%d",&blockSize[i]);
printf("Enter Process Sizes:\n");
for(i=0;i<np;i++)
scanf("%d",&processSize[i]);
for(i=0;i<np;i++)
allocation[i]=-1;
for(i=0;i<np;i++)
{
worstIdx=-1;
for(j=0;j<nb;j++)
{
if(blockSize[j] >= processSize[i])
{
if(worstIdx==-1 || blockSize[j] > blockSize[worstIdx])
worstIdx=j;
}
}
if(worstIdx!=-1)
{
allocation[i]=worstIdx;
blockSize[worstIdx]-=processSize[i];
}
}
printf("\nProcess No\tProcess Size\tBlock No\n");
for(i=0;i<np;i++)
{
printf("%d\t\t%d\t\t",i+1,processSize[i]);
if(allocation[i]!=-1)
printf("%d\n",allocation[i]+1);
}

```

```

else
printf("Not Allocated\n");
}
return 0;
}

```

Output:

```

$ gcc worstfit.c -o worstfit
(kali@kali)-[~]
$ ./worstfit
Enter Number of Blocks: 5
Enter Number of Processes: 4
Enter Block Sizes:
100 500 200 300 600
Enter Process Sizes:
212 417 112 426

Process No    Process Size    Block No
1             212             5
2             417             2
3             112             5
4             426             Not Allocated

```

SHELL SCRIPT

```

#!/bin/bash

echo "Worst Fit Memory Allocation Demonstration"

blocks=(100 500 200 300 600)

processes=(212 417 112 426)

echo "Memory Blocks: ${blocks[@]}"

echo "Processes: ${processes[@]}"

echo "Allocation Performed Using Worst Fit"

```

Output:

```
(kali㉿kali)-[~]  
$ ./worstfit.sh  
Worst Fit Memory Allocation Demonstration  
Memory Blocks: 100 500 200 300 600  
Processes: 212 417 112 426  
Allocation Performed Using Worst Fit
```